

# Proofs are Programs

Induction

# Short Recap

## Inductive Datatypes

```
Inductive nat : Type :=  
  | 0  
  | S (n : nat).
```

## Recursive Functions

```
Fixpoint plus (n : nat) (m : nat) : nat :=  
  match n with  
  | 0 => m  
  | S n' => S (plus n' m)  
  end.
```

## Proofs by

```
Theorem plus_0_n : ... .  
Proof. ... Qed.
```

## Simplification

```
simpl. reflexivity.
```

## Rewriting

```
rewrite -> H.
```



<https://etc.ch/APs2>

# Quizzes





<https://etc.ch/APs2>

# Quiz Time

```
Definition orb (b1:bool) (b2:bool) : bool :=  
  match b1 with  
  | true => true  
  | false => b2  
end.
```

Theorem review1: (orb true false) = true.

# Quiz Time



<https://etc.ch/APs2>

```
Definition orb (b1:bool) (b2:bool) : bool :=  
  match b1 with  
  | true => true  
  | false => b2  
end.
```

Theorem review2: forall b, (orb true b) = true.

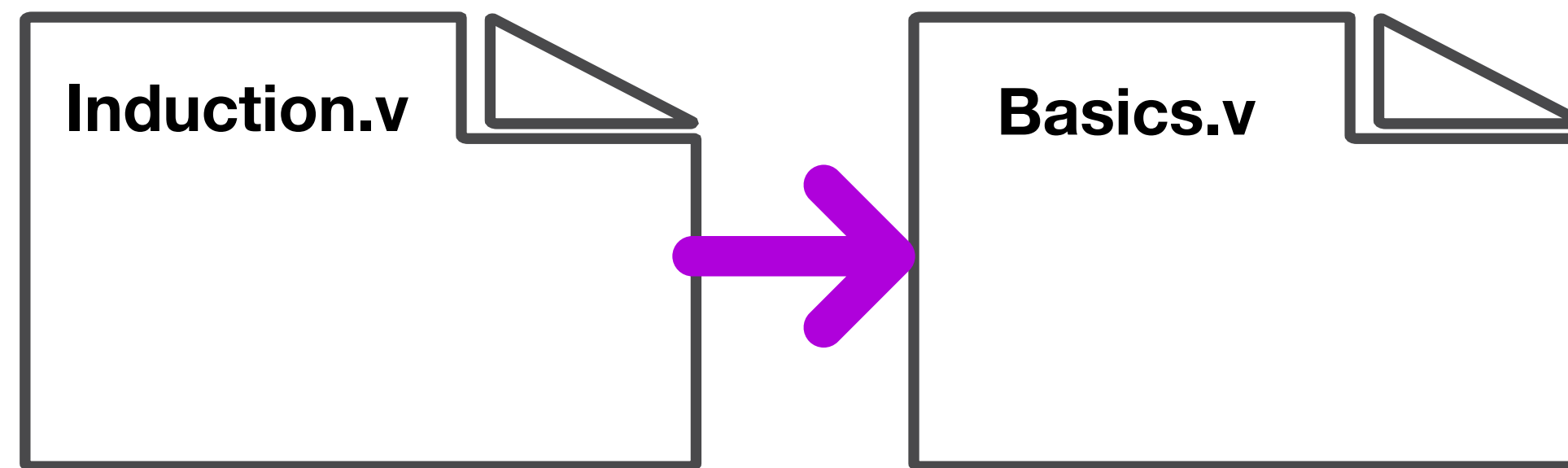
# What about this one?

```
Definition orb (b1:bool) (b2:bool) : bool :=  
  match b1 with  
  | true => true  
  | false => b2  
end.
```

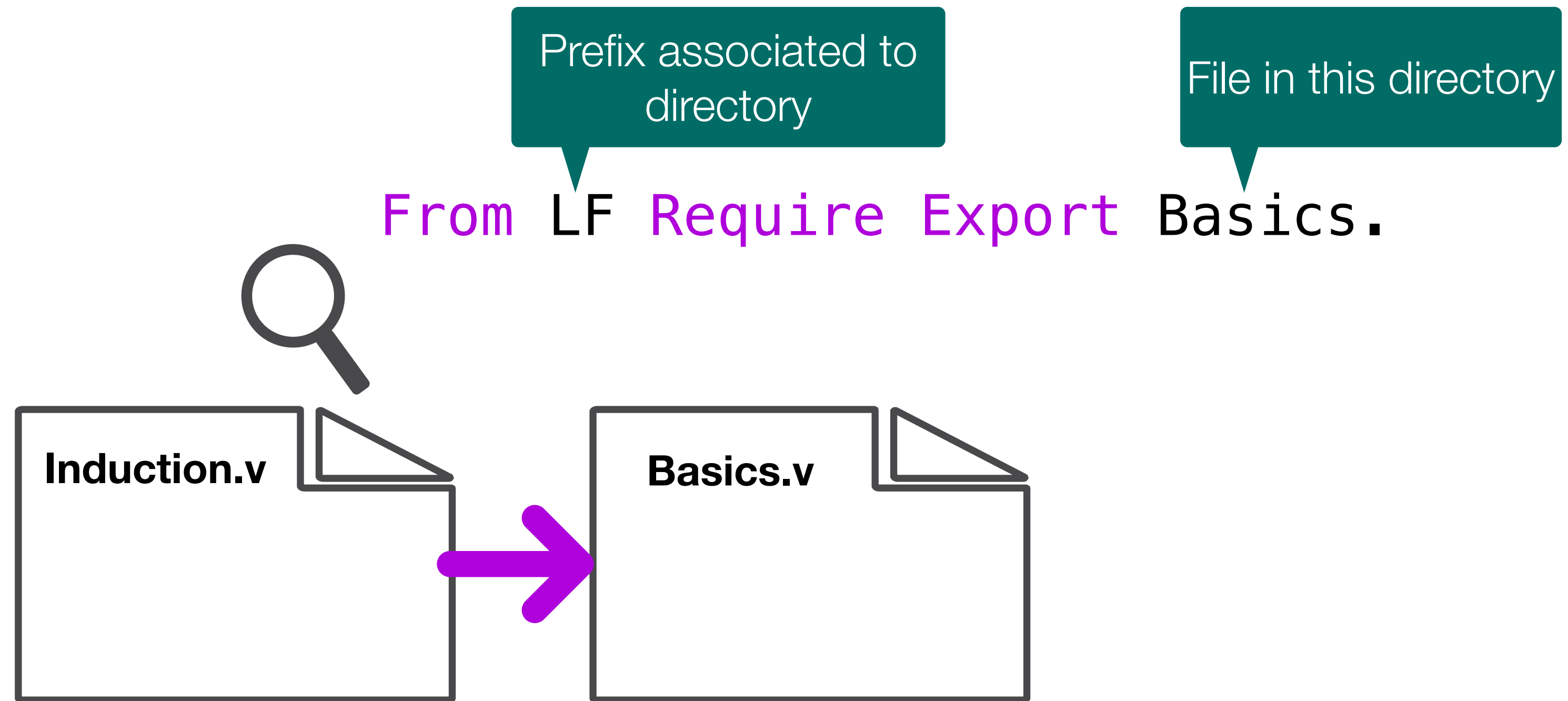
Theorem review3: forall b, (orb b true) = true.



# Building upon old files

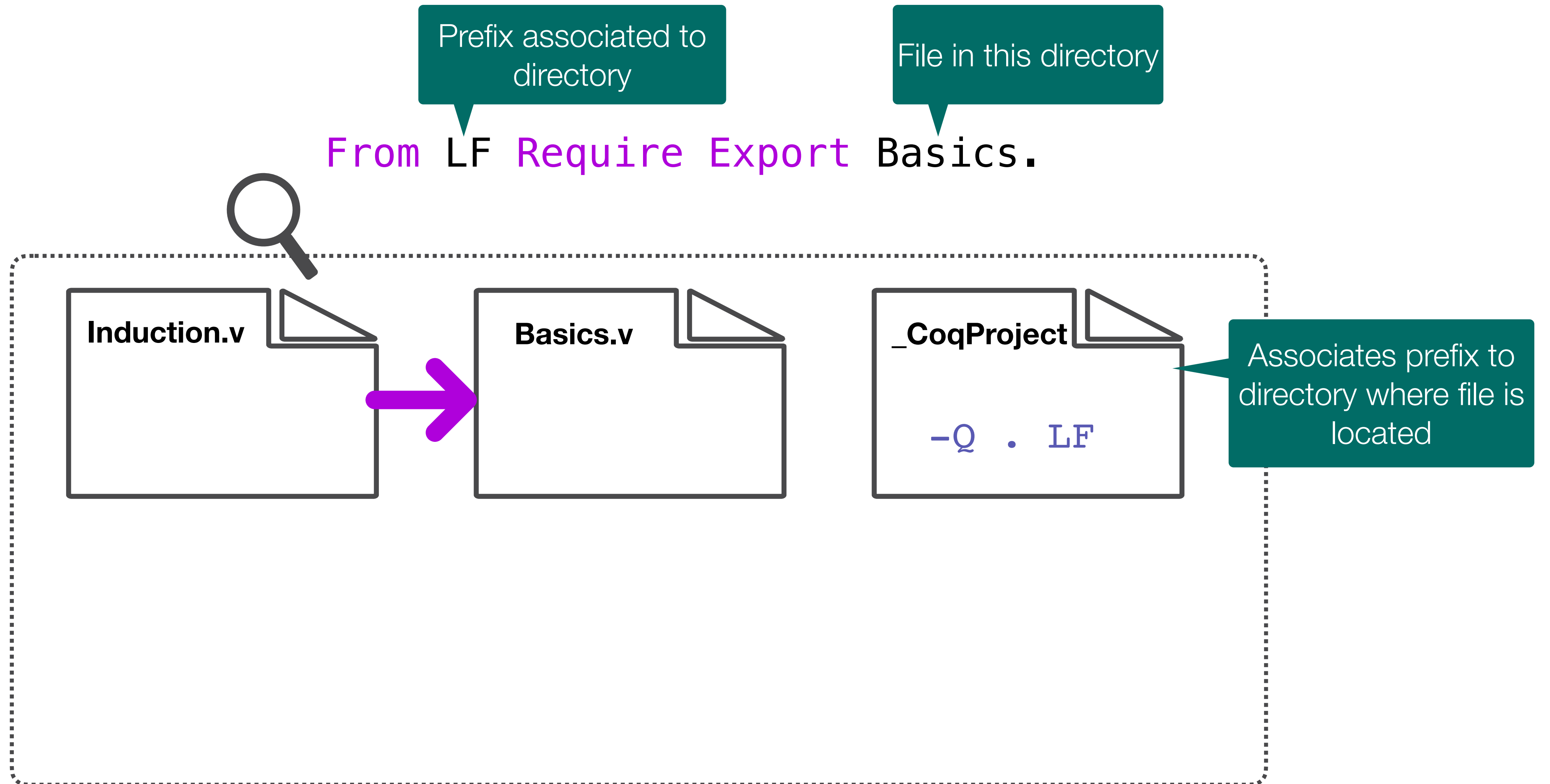


# Building upon old files

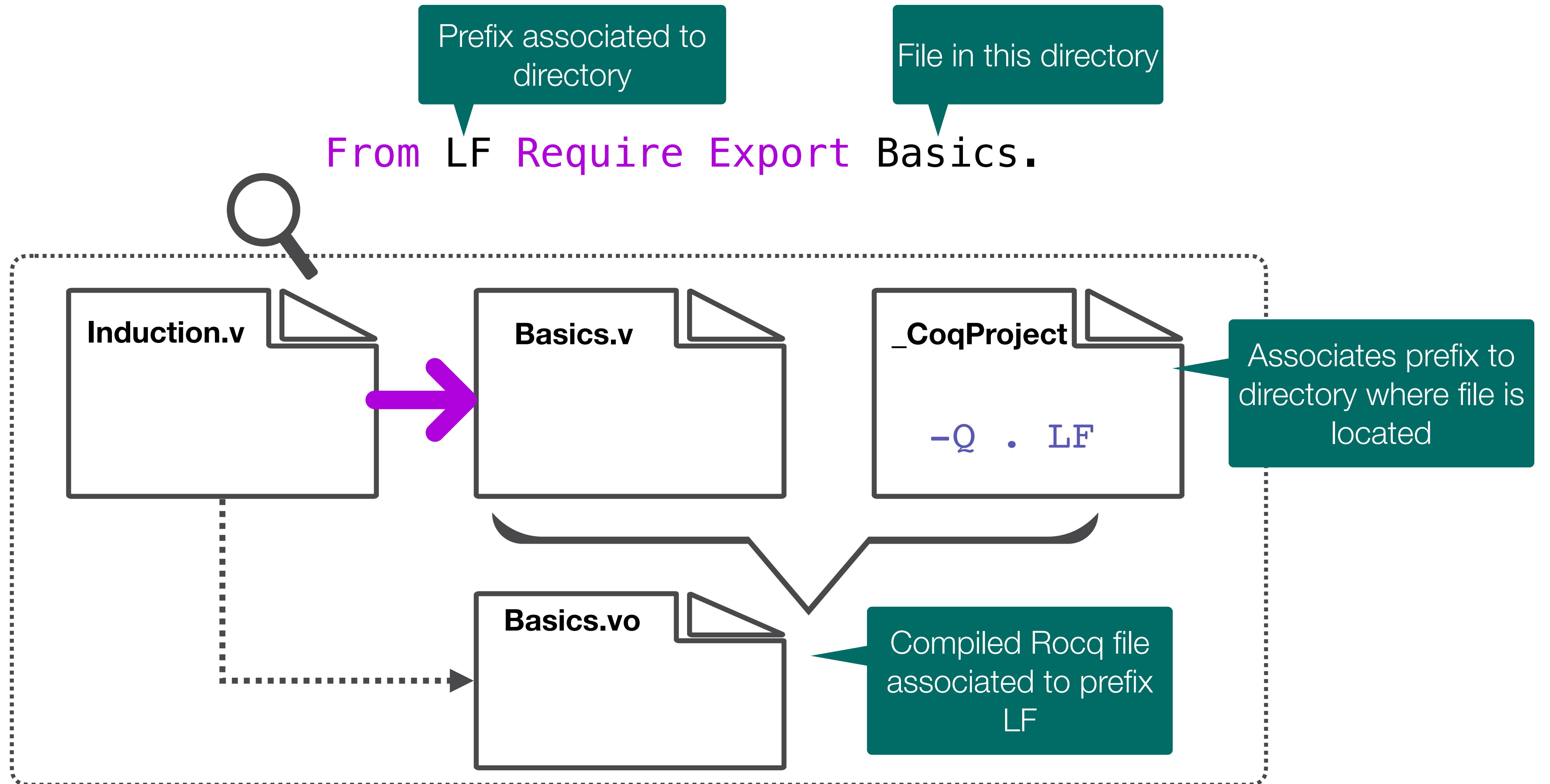




# Building upon old files



# Building upon old files



# Proofs by Case Analysis

**Theorem** `plus_1_neq_0` : `forall n : nat,`  
    `((n + 1) =? 0) = false.`



# Proofs by Case Analysis

Theorem `plus_1_neq_0` : forall n : nat,  
 (n + 1) =? 0) = false.

Theorem `plus_1_neq_0` : forall n : nat,  
 (eqb (plus n (S 0)) 0) = false.

# Proofs by Case Analysis

```
Theorem plus_1_neq_0 : forall n : nat,  
  (n + 1) =? 0 = false.
```

```
Theorem plus_1_neq_0 : forall n : nat,  
  (eqb (plus n (S 0)) 0) = false.
```

```
Fixpoint eqb (n m : nat) : bool :=  
  match n with  
  | 0 => match m with  
        | 0 => true  
        | S m' => false  
        end  
  | S n' => match m with  
            | 0 => false  
            | S m' => eqb n' m'  
            end  
  end.
```

```
Fixpoint plus (n : nat) (m : nat) : nat :=  
  match n with  
  | 0 => m  
  | S n' => S (plus n' m)  
  end.
```

# Proofs by Case Analysis

Theorem `plus_1_neq_0` : forall n : nat,  
((n + 1) =? 0) = false.

Theorem `plus_1_neq_0` : forall n : nat,  
(eqb (plus n (S 0)) 0) = false.

Does not simplify because plus is defined by recursion  
on first argument

```
Fixpoint eqb (n m : nat) : bool :=  
  match n with  
  | 0 => match m with  
        | 0 => true  
        | S m' => false  
        end  
  | S n' => match m with  
            | 0 => false  
            | S m' => eqb n' m'  
            end  
  end.
```

```
Fixpoint plus (n : nat) (m : nat) : nat :=  
  match n with  
  | 0 => m  
  | S n' => S (plus n' m)  
  end.
```



# Proofs by Case Analysis

# Proofs by Case Analysis

Theorem `plus_1_neq_0` : forall n : nat,  
 ((n + 1) =? 0) = false.

Proof.



# Proofs by Case Analysis

**Theorem** `plus_1_neq_0` : `forall n : nat,`  
`((n + 1) =? 0) = false.`

**Proof.**

`intros n.`

Let  $n$  be a natural number. We show that  $((n + 1) =? 0) = \text{false}$

# Proofs by Case Analysis

Theorem `plus_1_neq_0` : forall n : nat,  
((n + 1) =? 0) = false.

Proof.

`intros n.`

Let n be a natural number. We show that  $((n + 1) =? 0) = \text{false}$

`destruct n as [| n'] eqn:E.`

Case analysis on the natural number n

# Proofs by Case Analysis

**Theorem** `plus_1_neq_0 : forall n : nat,`  
`((n + 1) =? 0) = false.`

**Proof.**

`intros n.`

Let  $n$  be a natural number. We show that  $((n + 1) =? 0) = \text{false}$

`destruct n as [| n'] eqn:E.`

Case analysis on the natural number  $n$

`- reflexivity.`

Case 1) Let  $n = 0$ . Then by the definition of `plus` and `eqb`,  $((0 + 1) =? 0)$  evaluates to `false`.



# Proofs by Case Analysis

**Theorem** `plus_1_neq_0 : forall n : nat,`  
`((n + 1) =? 0) = false.`

**Proof.**

`intros n.`

Let  $n$  be a natural number. We show that  $((n + 1) =? 0) = \text{false}$

`destruct n as [| n'] eqn:E.`

Case analysis on the natural number  $n$

`- reflexivity.`

Case 1) Let  $n = 0$ . Then by the definition of `plus` and `eqb`,  $((0 + 1) =? 0)$  evaluates to `false`.

`- simpl.`

Case 2) Let  $n = S\ n'$  (E). Then by the definition of `plus` and `eqb`,  
 $((S\ n' + 1) =? 0) ==> (S\ (n' + 1) =? 0) ==> \text{false}$

# Proofs by Case Analysis

**Theorem** `plus_1_neq_0` : `forall n : nat,`  
`((n + 1) =? 0) = false.`

**Proof.**

`intros n.`

Let  $n$  be a natural number. We show that  $((n + 1) =? 0) = \text{false}$

`destruct n as [| n'] eqn:E.`

Case analysis on the natural number  $n$

– `reflexivity.`

Case 1) Let  $n = 0$ . Then by the definition of `plus` and `eqb`,  $((0 + 1) =? 0)$  evaluates to `false`.

– `simpl.`

Case 2) Let  $n = S\ n'$  (E). Then by the definition of `plus` and `eqb`,  
 $((S\ n' + 1) =? 0) ==> (S\ (n' + 1) =? 0) ==> \text{false}$

`reflexivity.`

By the reflexivity of equality, it holds that  $m+m = m+m$  is true

# Proofs by Case Analysis

**Theorem** `plus_1_neq_0` : `forall n : nat,`  
`((n + 1) =? 0) = false.`

**Proof.**

`intros n.`

Let  $n$  be a natural number. We show that  $((n + 1) =? 0) = \text{false}$

`destruct n as [| n'] eqn:E.`

Case analysis on the natural number  $n$

`- reflexivity.`

Case 1) Let  $n = 0$ . Then by the definition of `plus` and `eqb`,  $((0 + 1) =? 0)$  evaluates to `false`.

`- simpl.`

Case 2) Let  $n = S\ n'$  (E). Then by the definition of `plus` and `eqb`,  
 $((S\ n' + 1) =? 0) ==> (S\ (n' + 1) =? 0) ==> \text{false}$

`reflexivity.`

By the reflexivity of equality, it holds that  $m+m = m+m$  is true

**Qed.**



# Introduction Patterns

```
Inductive rgb : Type :=  
  | red  
  | green  
  | blue.
```

```
Inductive color : Type :=  
  | black  
  | white  
  | primary (p : rgb).
```

**c:** color

---

# Introduction Patterns

```
Inductive rgb : Type :=  
  | red  
  | green  
  | blue.
```

```
Inductive color : Type :=  
  | black  
  | white  
  | primary (p : rgb).
```

$c : \text{color}$

---

`destruct c.`

opens three subgoals  
(for all constructors of color)



# Introduction Patterns

```
Inductive rgb : Type :=  
  | red  
  | green  
  | blue.
```

```
Inductive color : Type :=  
  | black  
  | white  
  | primary (p : rgb).
```

`c: color`

---

`destruct c.`

opens three subgoals  
(for all constructors of color)

`destruct c as [].`

opens three subgoals

# Introduction Patterns

```
Inductive rgb : Type :=  
  | red  
  | green  
  | blue.
```

```
Inductive color : Type :=  
  | black  
  | white  
  | primary (p : rgb).
```

$c : \text{color}$

---

`destruct c.`

opens three subgoals  
(for all constructors of color)

`destruct c as [].`

opens three subgoals

`destruct c as [ | | ].`

opens three subgoals

# Introduction Patterns

```
Inductive rgb : Type :=  
  | red  
  | green  
  | blue.
```

```
Inductive color : Type :=  
  | black  
  | white  
  | primary (p : rgb).
```

c: color

destruct c.

opens three subgoals  
(for all constructors of color)

destruct c as [].

opens three subgoals

destruct c as [ | | ].

opens three subgoals

destruct c as [ | | p ].

opens three subgoals  
+ names argument of  
primary (p)



# Introduction Patterns

```
Inductive rgb : Type :=  
  | red  
  | green  
  | blue.
```

```
Inductive color : Type :=  
  | black  
  | white  
  | primary (p : rgb).
```

**c** : color

---

destruct c.

opens three subgoals  
(for all constructors of color)

destruct c as [].

opens three subgoals

destruct c as [ | | ].

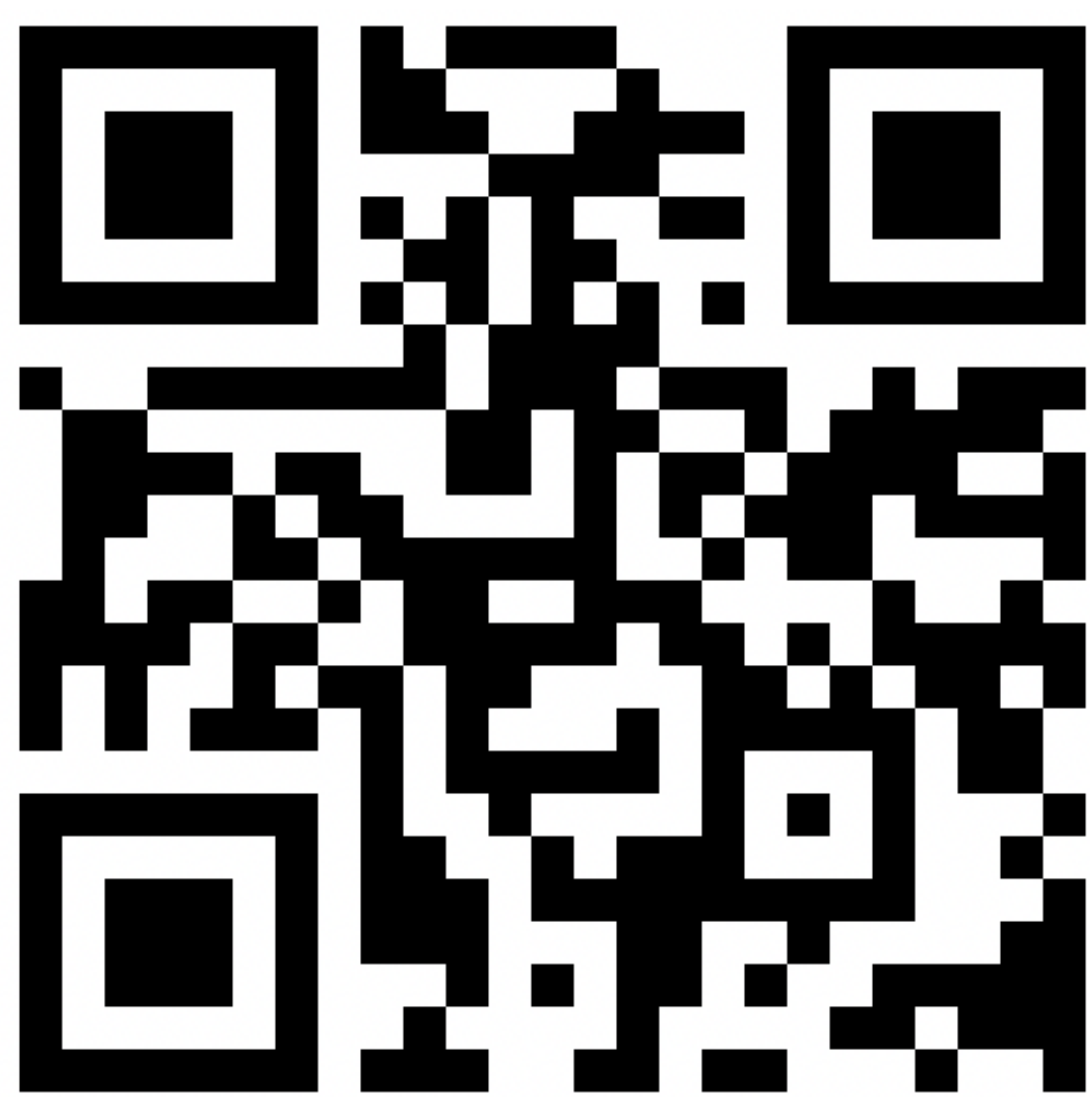
opens three subgoals

destruct c as [ | | p ].

opens three subgoals  
+ names argument of  
primary (p)

destruct c as [ | | [] ].

opens 5 subgoals:  
- black  
- white  
- primary red  
- primary green  
- primary blue

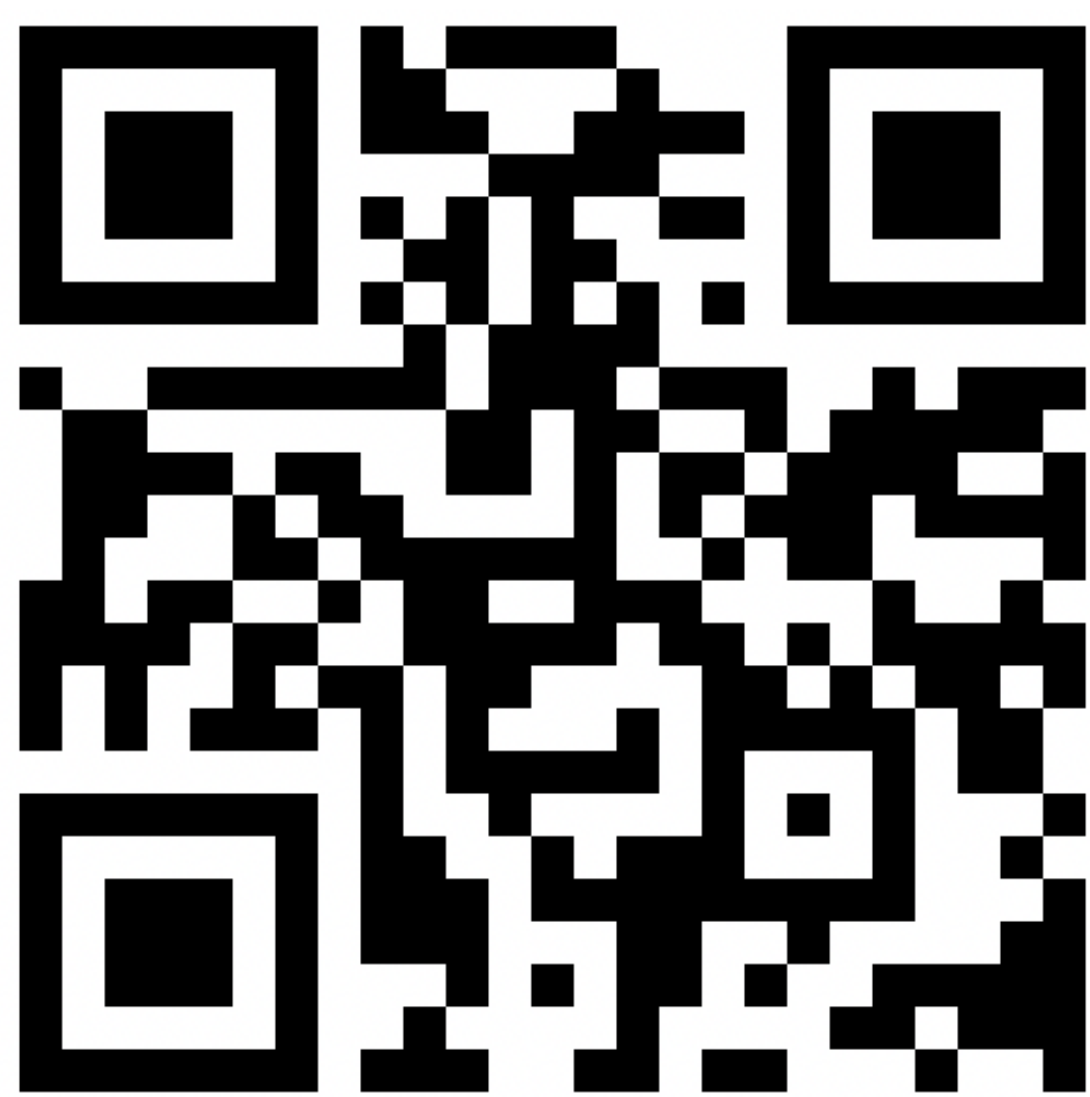


<https://etc.ch/APs2>

# Quiz time

```
Definition orb (b1:bool) (b2:bool) : bool :=  
  match b1 with  
  | true => true  
  | false => b2  
end.
```

Theorem review3: forall b, (orb b true) = true.



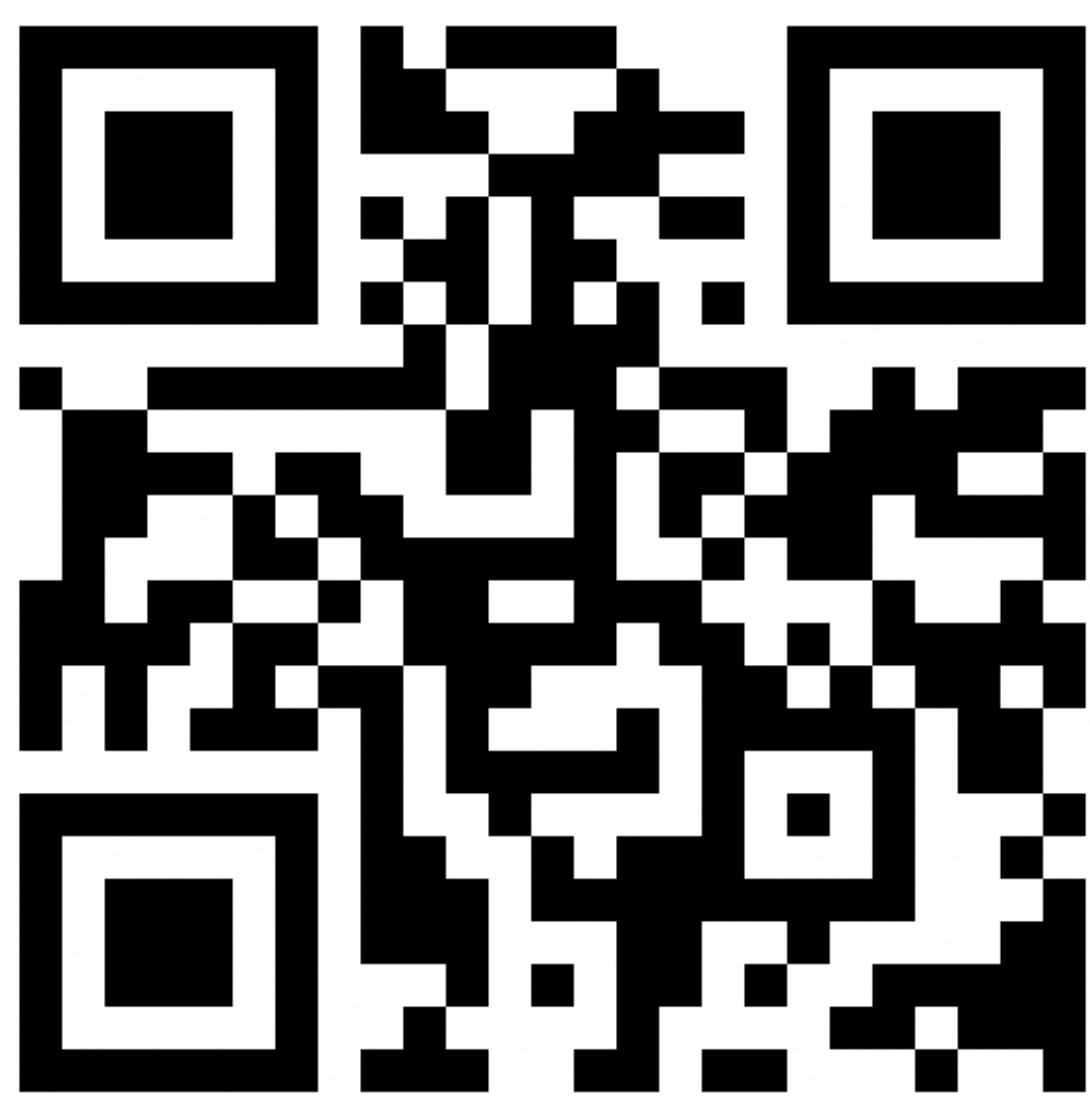
<https://etc.ch/APs2>

# Quiz time

```
Fixpoint plus (n : nat) (m : nat) : nat :=  
  match n with  
  | 0 => m  
  | S n' => S (plus n' m)  
end.
```

Theorem review4 : forall n : nat, n = 0 + n.





<https://etc.ch/APs2>

# Quiz time

```
Fixpoint plus (n : nat) (m : nat) : nat :=  
  match n with  
  | 0 => m  
  | S n' => S (plus n' m)  
end.
```

Theorem review5 : forall n : nat, n = n + 0.

# We reach limits easily...

Theorem `plus_0_n` : forall n:nat,  
0 + n = n.



Theorem `plus_0_r` : forall n:nat,  
n + 0 = n.



```
Fixpoint plus (n : nat) (m : nat) : nat :=  
  match n with  
  | 0 => m  
  | S n' => S (plus n' m)  
  end.
```



# Induction

- General principle to show that  $\forall n \in \mathbb{N}. P(n)$ 
  - show  $P(0)$
  - show that for any  $n > 0$  if  $P(n - 1)$  holds, then so does  $P(n)$ 
    - Alternatively show that for any  $n'$  if  $P(n')$  holds, then so does  $P(S\ n')$

- Typical example  $P(n) := \sum_{i=0}^n i = \frac{n(n+1)}{2}$

# Proof by Induction

# Proof by Induction

Theorem `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .



# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

`induction n as [| n' IHn'].`

Proof: by induction on  $n$ .

# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

`- (* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

`- (* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

`reflexivity.`

This follows directly from the definition of  $+$ .



# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + 0 = n'$  (IHn'). We must show that  $(S\ n') + 0 = S\ n'$

# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + 0 = n'$  (IHn'). We must show that  $(S\ n') + 0 = S\ n'$ .

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + 0) = S\ n'$ .



# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + 0 = n'$  (IHn'). We must show that  $(S\ n') + 0 = S\ n'$ .

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + 0) = S\ n'$ .

`rewrite -> IHn'.`

By the inductive hypothesis (IHn') this is equivalent to  $S\ n' = S\ n'$ .

# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + 0 = n'$  ( $IHn'$ ). We must show that  $(S\ n') + 0 = S\ n'$ .

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + 0) = S\ n'$ .

`rewrite -> IHn'.`

By the inductive hypothesis ( $IHn'$ ) this is equivalent to  $S\ n' = S\ n'$ .

`reflexivity.`

This concludes the proof.

# Proof by Induction

**Theorem** `add_0_r` : `forall n:nat,`  
`n + 0 = n.`

Theorem: For any  $n$ ,  $n + 0 = n$ .

**Proof.**

`intros n.`

Proof: by induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + 0 = 0$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + 0 = n'$  ( $IHn'$ ). We must show that  $(S\ n') + 0 = S\ n'$ .

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + 0) = S\ n'$ .

`rewrite -> IHn'.`

By the inductive hypothesis ( $IHn'$ ) this is equivalent to  $S\ n' = S\ n'$ .

`reflexivity.`

This concludes the proof.

**Qed.**



# Paper Proofs (Informal Proofs)

# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

`intros n m p.`  
`induction n as [| n' IHn'].`

Proof: By induction on  $n$ .



# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

`intros n m p.`

`induction n as [| n' IHn'].`

Proof: By induction on  $n$ .

`- (* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + (m + p) = (0 + m) + p$ .

# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

`intros n m p.`

Proof: By induction on  $n$ .

`induction n as [| n' IHn'].`

`- (* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + (m + p) = (0 + m) + p$ .

`reflexivity.`

This follows directly from the definition of  $+$ .



# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

`intros n m p.`

Proof: By induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + (m + p) = (0 + m) + p$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + (m + p) = (n' + m) + p$  ( $IHn'$ ).  
We must show that  $(S\ n') + (m + p) = ((S\ n') + m) + p$

# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

`intros n m p.`

Proof: By induction on  $n$ .

`induction n as [| n' IHn'].`

– `(* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + (m + p) = (0 + m) + p$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

– `(* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + (m + p) = (n' + m) + p$  (IHn').  
We must show that  $(S\ n') + (m + p) = ((S\ n') + m) + p$

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + (m + p)) = S\ ((n' + m) + p)$

# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

Proof: By induction on  $n$ .

`intros n m p.`

`induction n as [| n' IHn'].`

`- (* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + (m + p) = (0 + m) + p$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

`- (* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + (m + p) = (n' + m) + p$  (IHn').  
We must show that  $(S\ n') + (m + p) = ((S\ n') + m) + p$

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + (m + p)) = S\ ((n' + m) + p)$

`rewrite IHn'.`

which is immediate from the inductive hypothesis (IHn')

`reflexivity.`



# Paper Proofs (Informal Proofs)

**Theorem** `add_assoc''` : `forall n m p : nat,`  
`n + (m + p) = (n + m) + p.`

Theorem: For any  $n\ m\ p$ ,  
 $n + (m + p) = (n + m) + p$

**Proof.**

Proof: By induction on  $n$ .

`intros n m p.`

`induction n as [| n' IHn'].`

`- (* n = 0 *)`

First, suppose  $n = 0$ . We must show that  $0 + (m + p) = (0 + m) + p$ .

`reflexivity.`

This follows directly from the definition of  $+$ .

`- (* n = S n' *)`

Next, suppose  $n = S\ n'$ , where  $n' + (m + p) = (n' + m) + p$  (IHn').  
We must show that  $(S\ n') + (m + p) = ((S\ n') + m) + p$

`simpl.`

By the definition of  $+$  this follows from  $S\ (n' + (m + p)) = S\ ((n' + m) + p)$

`rewrite IHn'.`

which is immediate from the inductive hypothesis (IHn')

`reflexivity.`

**Qed.**

# Summary

## Proofs by

### Case Analysis

```
destruct n as [ | n'] eqn:E.
```

### Induction

```
induction n as [ | [n' H]].
```

## Introduction Patterns

```
destruct c as [ | | p ].
```

```
intros [ | | [] ].
```

## “Informal Proofs”



Proof: by induction on n.

## Proofs within Proofs

```
replace ... with ...  
{ ... }
```